# Arijit Bhatta

Gurugram, India • +91 7656956919 • arijitbhatta123@gmail.com • LinkedIn • GitHub

## PROFILE

Java Software Engineer with 2+ years of experience at NatWest Group, contributing to RESTful microservices that underpin retail and corporate banking operations. Well-versed across the backend stack — Spring Boot, JPA/Hibernate, shell scripting, and REST API design. Delivered features that reduced production incidents, automated 50+ batch jobs, and assisted in migrating legacy systems to microservices. Started as a self-taught developer, upskilled through Masai School, and joined NatWest directly in an engineering role. Currently focused on expanding expertise in distributed systems and cloud-native architecture.

## TECHNICAL SKILLS

**Languages:** Java, JavaScript, SQL, Bash
**Frameworks:** Spring Boot, Spring MVC, Spring Data JPA, Hibernate, Spring Security
**Databases:** MySQL, PostgreSQL
**Testing & CI/CD:** JUnit, Mockito, Jenkins, GitLab CI
**API & Web:** REST, JSON, HTTP, Swagger / OpenAPI
**Architecture:** Microservices, OOP, Design Patterns, Multithreading / Concurrency
**Tools & Platforms:** Git, Maven, Gradle, IntelliJ IDEA, Postman, Linux

## WORK EXPERIENCE

### Software Engineer | NatWest Group — Gurugram *Feb 2024 – Present*

- Working on a microservices-based banking platform (8+ services) processing ~100K+ daily transactions of fixed income bonds across retail and corporate banking — responsible for developing, maintaining, and improving multiple Spring Boot services.
- Personally own 3 Spring Boot microservices end-to-end — consistently delivering 2–3 production releases per sprint with zero rollbacks over the last 6 months through careful testing and staged deployments.
- Integrated with 5+ third-party REST APIs by defining clear request/response contracts, adding input validation and error handling, which reduced third-party integration issues by ~40% over two quarters.
- Tightened exception handling and business validation across Spring Boot services — contributed to a ~30% drop in production incidents in the services I own.
- Tuned JPA/Hibernate queries and transaction boundaries, bringing average API response times down by ~20% on the heaviest-used endpoints.
- Wrote and maintained Swagger/OpenAPI docs for all owned services, which noticeably reduced back-and-forth with partner teams during integration.
- Automated 50+ Linux batch jobs — deployments, log rotation, data exports — using shell scripts, saving several hours of manual ops work each week.
- Part of a 7-member Agile squad; worked directly with POs, BAs, QA, and architects to ship compliant features across two-week sprints.
- Write unit and integration tests using JUnit and Mockito for all owned services, maintaining ~80% test coverage — also review PRs for teammates and flag edge cases that slip past initial implementation.
- Took end-to-end ownership of 10+ features from requirement clarification through deployment and post-release monitoring, coordinating directly with QA and BAs without needing senior oversight.
- Helping migrate legacy monolith components to Spring Boot microservices — so far contributed to 3 service extractions, reducing code complexity and making each service easier to deploy on its own.

### Learner Success Manager | Simplilearn — Bangalore *Jul 2023 – Feb 2024*

- First point of contact for 100+ learners facing technical issues — debugged coding problems, environment setup errors, and platform issues, then translated them into clear tickets for the engineering team.

- Tracked learner progress across cohorts using internal dashboards, flagged at-risk learners early, and coordinated with instructors to unblock them before deadlines slipped.
- Worked closely with engineering and product teams to raise, prioritise, and close platform bugs — helped clear a backlog of 30+ open issues in my first two months.

## PROJECTS

**Personal Finance Tracker API** — *Java · Spring Boot · Spring Security · MySQL · Swagger*
*GitHub:* **[Link]**

- Built a REST API for tracking personal income, expenses, and budgets with JWT-based auth via Spring Security — moved away from session UUIDs here to get hands-on with stateless security properly.
- The schema design was the trickiest part — built it around monthly budget cycles with rollover logic so users could see overspending patterns across months without any manual resets.
- Documented every endpoint with Swagger UI; covers expense categorisation, budget alerts, and monthly summaries — kept the endpoints consistent and easy to follow throughout.

**COVID Vaccination Scheduler** — *Java · Spring Boot · MySQL · Hibernate*
*GitHub:* **[Link]**

- REST API for individuals to check vaccine availability and book slots — the main challenge was concurrent bookings, fixed by wrapping slot creation in Hibernate transactions so two users couldn't grab the same slot.
- Kept auth simple with session UUIDs for both admin and user flows — this was a prototype so skipping OAuth made sense, but I documented where that would need to change for production.
- Admin panel had full CRUD for centres, inventory, and appointments; added optimistic locking on inventory rows specifically to stop doses being over-allocated under load.

**Online Shopping Platform** — *Java · Spring Boot · MySQL · Hibernate*
*GitHub:* **[Link]**

- E-commerce REST API for buyers, sellers, and admins — the cart-to-order flow was the interesting part, keeping it consistent when users changed payment methods halfway through checkout took a few iterations to get right.
- Split payment handling into its own service layer so adding UPI, credit card, or net banking didn't require touching order logic — that separation saved a lot of pain when requirements changed mid-build.
- Wrapped every order and payment state change in a DB transaction — a failed payment leaves no trace on the order, which was non-negotiable for something handling real money flows.

**Task Management System** — *Java · Spring Boot · MySQL · Hibernate · HTML · CSS · JS*
*GitHub:* **[Link]**

- Full-stack task manager with a Spring Boot backend and plain JS frontend — skipped React on purpose to understand how data flow and state actually work before adding a framework on top.
- Filtering by title, assignee, status, and due date all runs through JPA Criteria queries — building a clean multi-condition query builder with optional filters was good practice.
- Frontend stays in sync via polling rather than websockets — not perfect, but straightforward to implement and debug. Learned pretty quickly where polling breaks down when updates get frequent.

## EDUCATION

**BSc Physics (CGPA 7.3/10)** | Balikuda College — Odisha, India *Jul 2017 – Nov 2020*

## CERTIFICATIONS

**Full Stack Web Development —** Masai School *(6-month intensive program — Java, Spring Boot, databases, full-stack development)*

**AWS Certified Developer – Associate**

**Oracle Java SE 17 Developer (1Z0-829)**